ParkinSense 2.0 — Full Project Plan

A screening, communication, and progress-tracking platform for Parkinson's patients

1. What we're building

A system with three parts:

1. **Screen** — detect signs of Parkinson's from a voice sample and a spiral/wave drawing (research-grade screening, not a medical diagnosis).
2. **Communicate** — speech-to-text, text-to-speech, and handwriting-to-text, so patients with unclear speech or shaky handwriting can still get their message across.
3. **Track** — log every session's scores over time so a patient or clinician can see a trend, not just one number.

We always call this **screening / decision support**, never diagnosis, and **progress tracking**, never a cure or healing.

2. Why this version is different from the AI-generated draft

The old version wrapped pretrained models (Whisper, a generic image classifier) with no training of our own and no accuracy numbers. This version trains real classifiers on real public datasets, reports accuracy/WER/CER honestly, and is simple enough for a 4-person team at 5th-semester level to actually finish.

3. Tech choices (kept deliberately simple)

- **One language: Python.** No Docker, no job queues, no React.
- **App:** Streamlit (turns a Python script into a web app with almost no front-end code).
- **Training:** Google Colab (free GPU, nothing to install).
- **Storage:** a single SQLite file (one file on disk, zero setup) or even a CSV for v1.
- **Models:** scikit-learn (SVM/Random Forest) for detection, `faster-whisper` for speech-to-text, `gTTS` or the browser's built-in voice for text-to-speech, EasyOCR for handwriting.

4. Team roles

Person	Owns
You (lead)	Repo setup, the Streamlit app shell, stitching everyone's code together, text-to-speech, the progress dashboard, the final demo
Member A	Speech: Parkinson's-vs-healthy voice classifier, speech-to-text
Member B	Writing: spiral/wave drawing classifier, handwriting-to-text (OCR)
Member C	App screens: recording page, drawing canvas, results page, and (with you) the dashboard

5. The golden rule: fake it before you make it

In Week 1, you (the lead) build the whole app with **dummy functions** — `detect_speech()` just returns a random score, `transcribe()` just returns a fixed sentence. The app *runs* and looks complete from day one. Over the next 11 weeks, each person's only job is to replace their one fake function with a real one. This is why last time you ended up building everything the night before — there was no working skeleton to plug into.

6. 12-week plan, step by step per person

Each step is 1–3 weeks. "Done when" tells you exactly how to know you're finished — no guessing.

You (Lead)

1. **Week 1 — Skeleton.** Create the GitHub repo and a `app.py` Streamlit app with 4 pages (Home, Speech Test, Writing Test, Progress) all using fake functions. Done when the app runs on your laptop and teammates can open it too.
2. **Weeks 2–3 — Glue code.** Define the exact function signatures everyone must match, e.g. `detect_speech(audio_file) -> (label, confidence)`. Share this as a one-page doc. Done when all 3 teammates have copied these signatures into their own notebooks.

3. **Weeks 4–6 — Storage.** Build the SQLite/CSV layer that saves every session (date, scores, transcript). Done when a dummy session saves and reloads correctly.
4. **Weeks 7–8 — Text-to-speech.** Add a "Speak this" button using gTTS, with the browser voice as backup. Done when typed text plays as audio in English and Hindi.
5. **Weeks 9–10 — Integration.** Replace every fake function with the real ones from A, B, C. Done when a live recording produces a real score and a real transcript.
6. **Weeks 11–12 — Dashboard & polish.** Build the trend chart (score over time) and a one-click PDF/summary export. Rehearse the demo twice. Done when you can run the full demo without touching code.

Member A — Speech

1. **Week 1 — Get the data.** Download MDVR-KCL (link below). Listen to 5 PD and 5 healthy clips so you understand the data. Done when files are organized into `pd/` and `healthy/` folders.
2. **Weeks 2–3 — Features.** In Colab, use the `librosa` library to extract jitter, shimmer, and MFCCs from each clip. Done when you have one spreadsheet: one row per recording, with features + label.
3. **Weeks 4–6 — Baseline model.** Train a simple SVM or Random Forest in scikit-learn on that spreadsheet. Done when you have an accuracy number and a confusion matrix.
4. **Weeks 7–8 — Make it honest.** Redo the split so the *same speaker* never appears in both train and test ("leave-one-speaker-out"). Done when you have an honest accuracy number, even if it's lower.
5. **Weeks 9–10 — Speech-to-text.** Run `faster-whisper` (small model) on 20–30 of your own voice recordings. Done when you have a Word Error Rate (WER) number.
6. **Weeks 11–12 — Package it.** Save your trained model as a `.pkl` file and write `detect_speech()` and `transcribe()` functions matching the lead's signatures. Done when the lead can call your functions directly.

Member B — Writing

1. **Week 1 — Get the data.** Download the Kaggle "Parkinson's Drawings" spiral/wave set (link below). Done when images are organized into `pd/` and `healthy/` folders.
2. **Weeks 2–3 — Look at the data.** Open 10 images from each class side by side and note visible differences (shakier lines, more pen lifts). Done when you can describe in 2 sentences what the model should be picking up on.

3. **Weeks 4–6 — Baseline model.** In Colab, fine-tune a small pretrained CNN (ResNet18) on the spiral images. Done when you have an accuracy number and a confusion matrix.
4. **Weeks 7–8 — Second dataset (optional stretch).** Try HandPD too and compare. Done when you know if your model still works on a dataset it hasn't seen.
5. **Weeks 9–10 — Handwriting-to-text.** Use EasyOCR on 20–30 samples of handwritten sentences (ask friends to write naturally, then shakily, for comparison). Done when you have a Character Error Rate (CER) number.
6. **Weeks 11–12 — Package it.** Save your trained model and write `detect_writing()` and `read_handwriting()` functions matching the lead's signatures. Done when the lead can call your functions directly.

Member C — App screens

1. **Week 1 — Recording page.** Build the Streamlit page that records a voice clip using `streamlit-webrtc` or a simple file upload as a fallback. Done when a recorded clip saves to disk.
2. **Weeks 2–3 — Drawing canvas.** Add a spiral-drawing canvas using `streamlit-drawable-canvas`. Done when a drawn spiral saves as an image.
3. **Weeks 4–6 — Results page.** Design how a score and transcript are displayed to the user (plain language, color-coded, not scary). Done when the lead approves the layout.
4. **Weeks 7–8 — Language switcher.** Add English/Hindi/Marathi labels across the app (a simple dictionary of translated strings is enough — no need for real-time translation). Done when switching language changes the visible text.
5. **Weeks 9–10 — Progress chart.** Build the line chart of scores over time using `st.line_chart`. Done when it displays dummy multi-session data correctly.
6. **Weeks 11–12 — Usability pass.** Get 3 people outside the team to use the app and note confusing points. Done when you've fixed the top 3 complaints.

7. Datasets (direct links)

Dataset	For	Link
MDVR-KCL (37 speakers, PD + healthy, English)	Speech detection	https://zenodo.org/records/2867216

Dataset	For	Link
Kaggle "Parkinson's Drawings" (spiral + wave)	Writing detection	https://www.kaggle.com/datasets/kmader/parkinsons-drawings
HandPD (736 images, spirals + meanders)	Writing detection (extra)	https://wwwp.fc.unesp.br/~papa/pub/datasets/Handpd/
UCI Parkinson's Telemonitoring (42 patients, 6 months, voice + clinician scores)	Progress-tracking demo data	https://archive.ics.uci.edu/dataset/189
UCI Spiral Drawings (graphics tablet, 62 PD + 15 healthy)	Writing detection (extra)	https://archive.ics.uci.edu/dataset/395

Rule: never mix PD samples from one dataset with healthy samples from another dataset — the model will learn the recording setup, not the disease.

8. Free tools / APIs

- **`faster-whisper`** — free, local, multilingual speech-to-text.
- **Browser `speechSynthesis`** — free, built into any browser, zero setup text-to-speech.
- **`gTTS`** (`pip install gTTS`) — free text-to-speech via Google Translate's voice; unofficial, so keep the browser voice as backup.
- **EasyOCR** — free handwriting/text recognition, supports Hindi.
- **Bhashini / ULCA** (https://bhashini.gov.in) — government Indic-language speech & translation APIs; register now, but don't depend on approval arriving in time for the demo.

9. Evaluation — what to report at the end

- Speech detection: accuracy, sensitivity, specificity, with a leave-one-speaker-out split (not random split).

- Writing detection: accuracy and a confusion matrix, tested on a second dataset if time allows.
- Speech-to-text: Word Error Rate (WER) before vs after any tuning.
- Handwriting-to-text: Character Error Rate (CER).
- Say plainly that results are indicative given small datasets (37–66 people), not clinical-grade.

10. Risks and how we handle them

- **Hospital data takes time to arrange.** Keep the whole project runnable on public data; treat hospital data as a bonus, not a dependency.
- **Datasets too big / not in English / broken links** — every dataset above has been checked: small, free, and directly downloadable.
- **Team feels overwhelmed** — every person has one function to write, one dataset, one number to report. No one touches another person's code until Week 9.
- **Running out of time before the demo** — the Week 1 skeleton means there is *always* a working app to show, even if some functions are still fake.

11. Milestones (demo every 2 weeks)

- **End of Week 3:** app skeleton + both raw datasets loaded and explored.
- **End of Week 6:** both baseline classifiers working with a first accuracy number.
- **End of Week 8:** speech-to-text and text-to-speech working end to end.
- **End of Week 10:** everything integrated — real scores, real transcripts, real voice output.
- **End of Week 12:** polished demo, dashboard, and final report.

12. One-line pitch for the presentation

"ParkinSense 2.0 screens for Parkinson's signs from voice and handwriting, helps patients with unclear speech or shaky writing communicate, and tracks their progress over time — built and evaluated on public clinical datasets, with every number backed by code we wrote and tested ourselves."